The final case applies when you're building an ASP.NET Core application that primarily runs background processing services, handling messages from a queue for example, but doesn't handle HTTP requests. The minimal hosting `WebApplication` and `WebApplicationBuilder` are, as their names imply, focused on building web applications, so they don't make sense in this situation.

NOTE You'll learn how to create background tasks and services using the generic host in chapter 34. .NET 8 introduces a non-HTTP version of the `WebApplicationBuilder` called `HostApplicationBuilder` which aims to simplify app bootstrapping for your background services.

If you're not in any of these situations, strongly consider using the minimal hosting `WebApplication` approach and the imperative, scriptlike bootstrapping of top-level statements.

NOTE The fact that you're using `WebApplication` doesn't mean you have to dump all your service and middleware configuration into Program.cs. For alternative approaches, such as using a `Startup` class you invoke manually or local functions to separate your configuration, see my blog post at http://mng.bz/0KKJ.

In this chapter I provided a relatively quick overview of the generic host and Startup-based approach. If you're thinking of moving from the generic host to minimal hosting, or if you're familiar with minimal hosting but need to work with the generic host, you may find yourself looking around for an equivalent feature in the other hosting model. The documentation for migrating from .NET 5 to .NET 6 provides a good description of the differences between the two models, and how each individual feature has changed. You can find it at http://mng.bz/KeeX.